

표적 궤적 모의

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그리기까지

개념 · 좌표변환 코드 이용 · 표적 구조체 · 궤적 모의 구현 · 결과

레이다 시스템 소프트웨어 스터디 · Part 3

INDEX

목차

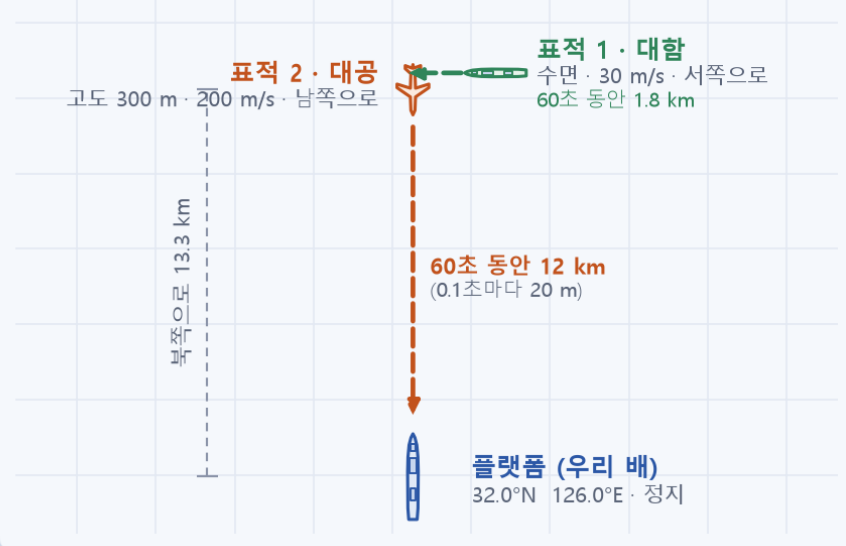
00	과제 요구사항	네 가지 제출 항목과 이 자료의 대응
01	필요한 개념	좌표계 · 위치와 속도 시간 적분 · 기동 모델
02	좌표변환 코드 이용	함수 넷 · 인자 순서와 단위 프로그램 구성
03	표적 구조체 정의	구조체 셋 · 항목 ↔ 칸 값 채우기
04	궤적 모의 구현	다섯 단계 · 함수별 코드 결과 화면
05	시뮬레이션 결과	실행 화면 · 궤적과 고도 기동 · 검증

02 ~ 05 가 과제 제출 항목 (1) ~ (4) 에 그대로 대응합니다. 개념은 코드를 읽는 데 필요한 만큼만.

00. 과제 3 요구사항과 이 자료의 대응

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라. 제출 항목 넷은 각각 한 장(章)으로

위에서 내려다본 그림 (대략 축척)



시뮬레이션 시간

60 초 · 0.1 초 간격

- $t = 0.0 \text{ s}$
- $t = 0.1 \text{ s}$
- $t = 0.2 \text{ s}$
- $t = 0.3 \text{ s}$
- ⋮
- $t = 60.0 \text{ s}$

위치 갱신 600 번
표적마다 · 위경도로 기록

구조체 · 갱신 · 출력 조건

표적 최대 10 개, 표적마다 기동 최대 30 개

기동 = Gravity Value · TurnType (0 없음 · 1 Roll · 2 Yaw · 3 Pitch) · StartTime · EndTime

위치 · 속도는 ECEF 에서 갱신, 결과는 LLA 로 출력하고 그림으로

플랫폼은 정지 (속력 0). 이 프로그램은 플랫폼도 같은 표적 구조체로 다룬다

과제가 제출하라고 한 것

이 자료

(1) 좌표변환 코드 이용	02 장 · 10 ~ 12
(2) 표적 구조체 정의 설명	03 장 · 14 ~ 16
(3) 궤적 모의 구현 방법을 코드로	04 장 · 18 ~ 23
(4) 시뮬레이션 결과 설명	05 장 · 25 ~ 28

시뮬레이션 조건

플랫폼 32.0°N 126.0°E, 정지

대함 표적 32.125°N 126.03°E, 고도 0 m, 30 m/s, yaw 270° (서쪽)

대공 표적 32.12°N 126.0°E, 고도 300 m, 200 m/s, yaw 180° (남쪽)

시간 60 s, 간격 0.1 s

자세각은 roll · pitch · yaw 전부 0° 에서 시작. 기동 목록은 과제에 없어 예시를 정해 넣었다 (8장).

필요한 개념

- 1 좌표계 넷, 그리고 왜 ECEF 인가
- 2 위치와 속도는 다르게 변환한다 — 회전 두 번
- 3 시간 적분과 지구 곡률 — 60초 직진하면 11 m 떠오른다
- 4 기동 모델 — 하중배수 · 세 축 · 시간표

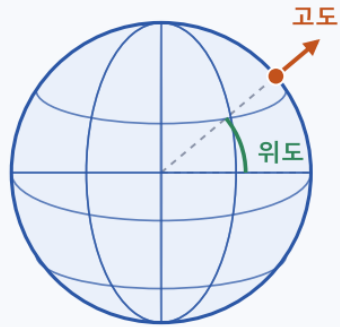
약속 : 각도는 도($^{\circ}$) 로 말하고 프로그램 안에서는 라디안. 거리는 m, 시간은 s.

01. 좌표계 넷, 그리고 왜 ECEF 인가

입력·출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체

LLA 위도 · 경도 · 고도

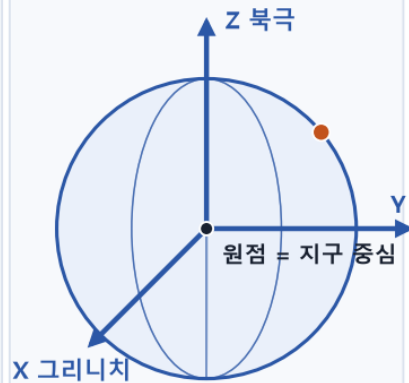
"지도 주소" — 사람이 읽는 자리



경도는 자오선에서 잴 각
고도는 타원체 면에서 잴 높이

ECEF 지구 중심 xyz

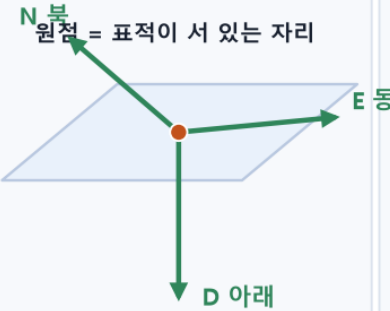
"지구에 박힌 자" — 계산하는 자리



지구와 함께 도는 직교좌표 (m)

NED 북 · 동 · 아래

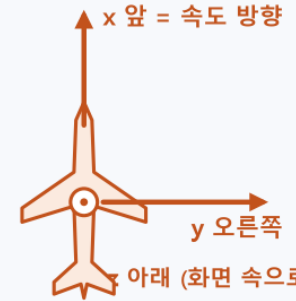
"내 발밑 나침반" — 수평면 기준



표적이 움직이면 같이 따라간다

동체 앞 · 오른쪽 · 아래

"표적의 몸" — 속도가 적힌 자리



속력 V 는 늘 x 축 위에만 있다

이번 과제에서의 역할

LLA

초기값을 받고, 매 스텝 결과를 적는 형식

ECEF

위치와 속도를 갱신하는 자리

NED

표적 자리의 수평면. 자세각의 기준이자 속도 방향을 만드는 자리

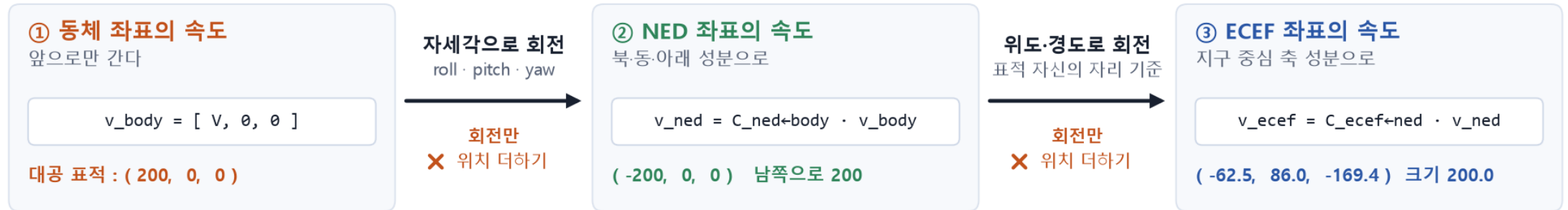
동체

속력 V 가 적힌 자리. 앞으로 V , 옆·아래는 0

왜 ECEF 인가 : 위도 $1^\circ = 111 \text{ km}$, 경도 $1^\circ = 94 \text{ km}$ (32°N) — 각도에 미터를 더할 수 없다. ECEF 는 세 축이 미터라 $p + v \cdot \Delta t$ 한 줄 플랫폼과 무관한 절대 좌표라, 나중에 배가 움직여도 표적 궤적이 흔들리지 않는다.

01. 위치와 속도는 다르게 변환한다 — 회전 두 번

위치는 원점이 있어야 말이 되고, 속도는 화살표 하나면 된다. 그래서 속도는 회전만



속도는 화살표라서 두 번 다 회전만 한다. ③ 에서 위도·경도는 플랫폼이 아니라 표적이 지금 있는 자리의 값이다.

위치 : $\mathbf{p}_{\text{new}} = \mathbf{R} \cdot \mathbf{p} + \mathbf{t}$

원점이 다르면 숫자가 다르다.
회전하고 원점 차이만큼 옮긴다 (과제 2).

속도 : $\mathbf{v}_{\text{new}} = \mathbf{R} \cdot \mathbf{v}$

어디에 그러도 같은 화살표.
축이 돌아간 만큼만 돌린다. $\mathbf{t}$ 를 더하면 속도가 아니게 된다.

검산

결과 벡터의 크기는 항상 V (200.000).
플랫폼 위치를 잘못 더하면 6,372 km/s.

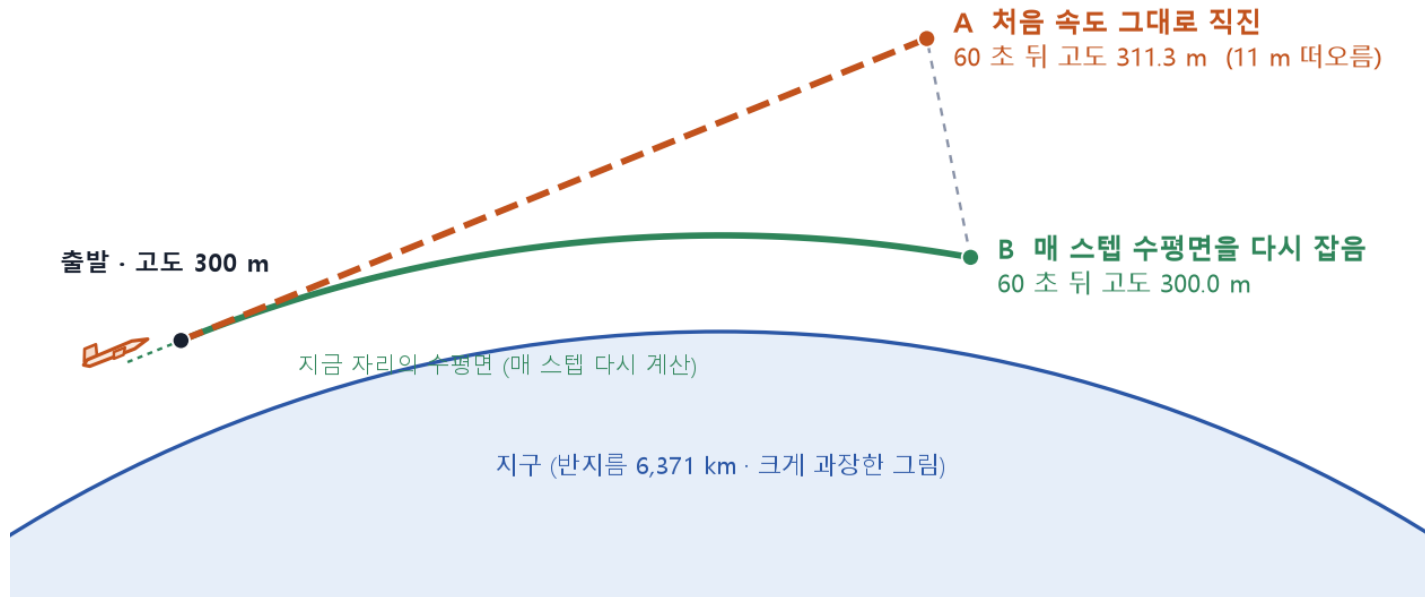
과제의 "추가 고민" (동체 속도 $\mathbf{V}_{\text{heading}} \rightarrow \text{ECEF}$) 의 답 : 자세각으로 한 번, 표적 자신의 위경도로 한 번, 평행이동은 없다

$\mathbf{f}_{\text{Trans_Ned_To_Ecef}}$ 는 플랫폼 위치를 더하게 되어 있으므로, 속도를 넣을 때는 그 인자에 0 을 준다.

01. 시간 적분과 지구 곡률

새 위치 = 지금 위치 + 속도 × 0.1 초. 단, 매 스텝 수평면을 다시 잡지 않으면 60초에 11 m 떠오른다

왜 떠오르나 : 지구가 둥글어서 접선 방향으로 12 km 가면 땅이 11 m 아래로 내려간다 ($d^2 / 2R$)
그래서 매 스텝 표적의 현재 위경도로 NED 수평면을 다시 잡고, 그 위에서 속도 방향을 다시 만든다



시간 적분 : $p(t+\Delta t) = p(t) + v(t) \cdot \Delta t$

ECEF x, y, z 에 각각. $\Delta t = 0.1$ s.

한 스텝 안에서 속도가 일정하니 이 한 줄이면 정확.

실험 (대공 표적 60초 직진)	60초 뒤 고도
A 처음 ECEF 속도를 60초 내내 그대로	311.3 m
B 매 스텝 현재 위경도로 NED 를 다시 잡음	300.0 m

왜 11 m 인가

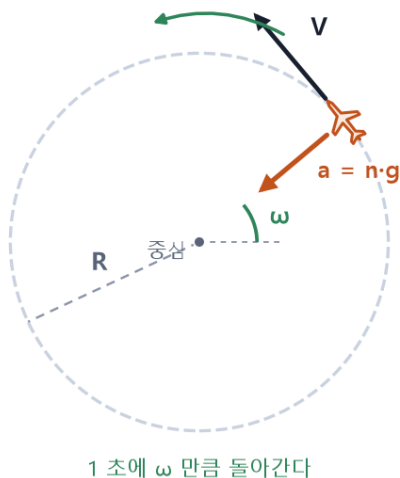
접선을 따라 d 만큼 가면 땅은 $d^2 / 2R$ 만큼 내려간다.
 $12,000^2 / (2 \times 6,371,000) = 11.3$ m. 실험값과 일치.

그래서 5단계 LLA 변환은 출력용만이 아니라 다음 스텝의 수평면(NED 축) 을 만드는 기준이다

위경도를 구해야 NED 축이 나오고, NED 축이 있어야 속도 방향이 나온다. 이 순서를 지키면 수평 비행이 수평으로 유지된다.

01. 기동 모델 — 하중배수, 세 축, 시간표

과제의 Gravity Value 는 "몇 g 로 도는가". 속력은 그대로, 방향만 바뀐다. TurnType 은 동체의 어느 축을 돌리는가



등속 원운동 공식

구심가속도

$$a = V^2 / R = n \cdot g$$

선회 반경

$$R = V^2 / (n \cdot g)$$

각속도

$$\omega = V / R = n \cdot g / V$$

$$g = 9.80665 \text{ m/s}^2$$

V 는 변하지 않는다

$$\omega = n \cdot g / V, \quad R = V^2 / (n \cdot g)$$

$a = V^2/R = n \cdot g$. 프로그램에서는 매 스텝 자세각 $\pm \omega \times 0.1 \text{ s}$ (3 g 면 한 스텝에 0.84°)

감 잡기

1 g 서 있을 때, 0.3 g 승용차 커브, 3 g 전투기 급선회, 9 g 한계. 각속도가 속력에 반비례 — 배 기동에는 0.1 g 같은 작은 값.

표적	하중배수	각속도	선회 반경	비고
대공 200 m/s	1 g	2.81 °/s	4,079 m	180° 에 64 s
대공 200 m/s	3 g	8.43 °/s	1,360 m	180° 에 21 s
대공 200 m/s	5 g	14.05 °/s	816 m	180° 에 13 s
대함 30 m/s	0.1 g	1.87 °/s	918 m	90° 에 48 s

TurnType	동체 축	효과	과제 예시
0	—	직진	기동 없음
1 Roll	x (앞뒤)	자세만 기울 · 방향 그대로	—
2 Yaw	z (위아래)	좌우 선회	대공 +3 g 10~20 s, 대함 -0.1 g 20~50 s
3 Pitch	y (날개)	상승 · 강하	대공 +2 g 35~40 s

시간표 규칙 셋

StartTime ≤ t < EndTime 이면 적용, 밖이면 직진. 부호 : n < 0 이면 반대 방향.

겹침 : 먼저 적힌 것 하나. 끝난 뒤 : 자세 유지 (안 건드리는 것이 곧 유지).

좌표 변환 코드 이용

- 1 참고 소스 코드와 이번에 쓰는 함수 넷
- 2 인자 순서와 단위 — 컴파일해서 확인한 규약
- 3 프로그램 구성

좌표 변환은 참고 소스 코드의 함수를 그대로 부르고, 인자 순서와 단위는 부르는 쪽에서 맞췄다.

02. 참고 소스 코드와 이번에 쓰는 함수 넷

좌표변환 14 함수 중 넷을 부른다. 초기화에 하나, 매 스텝 속도 변환에 둘, 매 스텝 출력에 하나

참고 소스 코드 (5 파일)

CoordinateTransform.c / .h

좌표변환 14 함수 (WGS-84, 각도 라디안)

matrixCalcLib.c / .h

3×3 행렬 곱 · 합 (변환 함수 안에서 쓰임)

Define.h

DOUBLE64 · INT32 · VOID, PI, G_FORCE 9.80665

함수 (인자)	하는 일	어디서
f_Trans_Lla_To_Ecef (Lon, Lat, Alt)	위경도·고도 → ECEF 위치	초기화 (f_InitTarget) 한 번
f_Trans_Body_To_Ned (x, y, z, Roll, Yaw, Pitch)	동체 벡터 → NED 벡터 (자세각 회전)	③ 속도 변환, 매 스텝
f_Trans_Ned_To_Ecef (n, e, d, Pf_X, Pf_Y, Pf_Z, Lat, Lon)	NED 벡터 → ECEF 벡터 (+ 플랫폼 위치)	③ 속도 변환, 매 스텝 (Pf = 0)
f_Trans_Ecef_To_Lla (X, Y, Z)	ECEF 위치 → 위경도·고도	⑤ 매 스텝 끝, 출력 + 다음 NED 기준

한 스텝에서 넷이 이어지는 순서 (초기화 한 번은 위경도 → f_Trans_Lla_To_Ecef → ECEF 위치)



나머지 열 함수 (안테나 계열 4 · ENU 2 · 플랫폼 보정 · ECEF→NED · Body↔Ant) 는 이번 과제에서 부르지 않는다

표적을 만드는 쪽이라 안테나 좌표계가 없고, 플랫폼이 서 있어 플랫폼 보정도 없다.

02. 인자 순서와 단위 — 컴파일해서 확인한 규약

헤더만 읽고 넘어가지 않고 하나씩 값을 찍어 봤다. 틀리면 어떤 값이 나오는지도 같이

함수	인자 순서 · 단위	틀리면
<code>f_Trans_Lla_To_Ecef</code>	경도, 위도, 고도 — 경도가 첫째 인자. 라디안 · m	위도·경도를 바꿔 넣으면 북위 54° 동경 212°
<code>f_Trans_Body_To_Ned</code>	(x, y, z) 다음 Roll, Yaw, Pitch — Roll-Pitch-Yaw 가 아니다	yaw 와 pitch 가 바뀌어 대공 표적이 땅으로 쏜다
<code>f_Trans_Ned_To_Ecef</code>	(n, e, d), 플랫폼 ECEF 위치 셋, 플랫폼 위도·경도 — 속도엔 위치에 0	플랫폼 위치를 넣으면 $ v = 6,372 \text{ km/s}$
<code>f_Trans_Ecef_To_Lla</code>	X, Y, Z [m] → Lat, Lon [rad], Alt [m]. 5회 고정 반복	반복 부족이면 고도 오차 — 우리 위치에서 왕복 $3e-5 \text{ m}$ 로 충분
공통	각도는 전부 라디안. DEG2RAD / RAD2DEG 매크로는 target_sim.h 에 정의	도(°) 를 그대로 넣으면 $32 \text{ rad} = 1,833^\circ$

target_sim.c 가 좌표변환 함수를 부르는 네 줄 (pstTgt-> 생략)

```
// 초기화 (f_InitTarget) : 경도가 첫째 인자
stPos = f_Trans_Lla_To_Ecef(stInitLla.Lon, stInitLla.Lat, stInitLla.Alt);

// ㉓ 동체 → NED (f_BodyVelToEcef) : 자세각 순서 Roll, Yaw, Pitch
stNed = f_Trans_Body_To_Ned(dVheading, 0.0, 0.0, stAtt.Roll, stAtt.Yaw, stAtt.Pitch);

// ㉔ NED → ECEF : 속도라서 플랫폼 위치 인자(가운데 셋) 에 0. 위경도는 표적 자신의 것
stVel = f_Trans_Ned_To_Ecef(stNed.x, stNed.y, stNed.z, 0.0, 0.0, 0.0, stLla.Lat, stLla.Lon);

// ㉕ ECEF → LLA (f_StepTarget) : 출력이자 다음 스텝의 NED 기준
stLla = f_Trans_Ecef_To_Lla(stPos.x, stPos.y, stPos.z);
```

컴파일해서 확인한 것

- ① Body→NED 행렬 = $R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$ 과 소수점 6자리 일치 (과제 2 규약)
- ② LLA → ECEF → LLA 왕복 오차 $3e-5 \text{ m}$
- ③ 속도 변환 결과 $|v| = 200.000000 = V$
- ④ Pf 를 넣으면 $6,372 \text{ km/s} \rightarrow \text{속도엔 } 0 \text{ 이 맞다}$

02. 프로그램 구성

좌표변환은 참고 소스 코드의 함수가 하고, C 코어가 표적을 움직이고, MFC 화면은 결과를 보여 준다

참고 소스 코드

C · 좌표변환

CoordinateTransform.c / .h

좌표변환 14 함수 (WGS-84)

matrixCalcLib.c / .h

행렬 곱 · 합

Define.h

타입 · 상수 (PI, G_FORCE)

링크



TargetSimCore (C 라이브러리)

이번에 짰 부분 · target_sim.h / .c

ST_Maneuver · ST_Target · ST_Scenario

구조체 셋

f_SetAssignment

과제 조건 채우기

f_StartScenario · f_StepScenario

초기화, 0.1 초 한 스텝

호출



TargetSimUI (MFC 대화상자)

돌리고 보여 주기만 · C++

[실행]

60 초를 돌려 위경도를 모은다

지도 · 슬라이더 · 재생

궤적과 임의 시각의 위치

표 · **[CSV 저장]**

위경도 출력

좌표변환은 전부 참고 소스 코드의 함수가 하고, 코어는 그 함수를 순서대로 부르지만 한다. MFC 는 계산을 하지 않는다.

코어의 함수 다섯	하는 일	부르는 곳
f_SetTarget	표적 카드 한 장의 입력 칸을 채운다 (각도는 도로 받는다)	f_SetAssignment
f_AddManeuver	기동 하나를 목록 끝에 붙인다	f_SetAssignment
f_SetAssignment	과제 조건 : 플랫폼 · 대함 · 대공, 60 s · 0.1 s	화면 [실행]
f_StartScenario	입력 칸으로 상태 칸을 채운다 (위경도 → ECEF)	화면 [실행], 한 번
f_StepScenario	플랫폼과 표적 전부를 0.1 초 진행한다	화면 [실행], 0.1 초마다

표적 구조체 정의

- 1 구조체 셋 — `target_sim.h`
- 2 과제 항목이 들어간 칸, 그리고 설계 결정
- 3 값 채우기 — `f_SetTarget` · `f_AddManeuver` · `f_SetAssignment`

입력 칸은 사람이 적고, 상태 칸은 프로그램이 갱신한다. 플랫폼도 카드 한 장.

03. 구조체 셋 — target_sim.h

위 다섯 칸이 입력(사람이 적는 값), 아래 네 칸이 상태(프로그램이 갱신). 좌표 타입은 참고 소스 코드의 구조체 그대로

표적 카드 #2

ST_Target

입력 (사람이 적는 칸)

초기 위치 위경도 · 고도로
32.12°N 126.0°E 300 m

동체 속도 앞 방향 하나로
200 m/s

초기 자세
roll 0° pitch 0° yaw 180°

기동 목록 (최대 30 개)

g	Type	Start	End
3	Yaw	10	20
2	Pitch	35	40
...			

초기화
→
한 번

상태 (프로그램이 갱신)

ECEF 위치
x, y, z [m]

ECEF 속도
vx, vy, vz [m/s]

현재 자세
roll, pitch, yaw

현재 위경도
출력용 · 매 스텝 변환



매 스텝 갱신 · 600 번
입력 칸은 건드리지 않는다

플랫폼도 속도 0 인 표적 카드

갱신 함수가 하나로 통일되고, 배가 움직이는 시나리오가 와도 카드 값만 바꾸면 된다. MAX_TARGET 10, MAX_MANEUVER 30.

```
typedef struct
{
    DOUBLE64    dGravity;        // 하중배수 [g]. 부호가 방향
    INT32        nTurnType;       // 0 없음 1 Roll 2 Yaw 3 Pitch
    DOUBLE64    dStartTime;      // [s]
    DOUBLE64    dEndTime;        // [s]
} ST_Maneuver;

typedef struct
{
    STRUCT_Coord_Lla    stInitLla;        /* 입력 */
    DOUBLE64            dVheading;        // 초기 위치 [rad, rad, m]
    STRUCT_Coord_Attitude stInitAtt;      // 동체 속도 [m/s]
    INT32                nManeuverCnt;     // 초기 자세 [rad]
    ST_Maneuver          astManeuver[MAX_MANEUVER]; // 기동 수
                                                    /* 상태 (ECEF 에서 갱신) */
    STRUCT_Coord_Rect    stPos;           // ECEF 위치 [m]
    STRUCT_Coord_Rect    stVel;           // ECEF 속도 [m/s]
    STRUCT_Coord_Attitude stAtt;          // 현재 자세 [rad]
    STRUCT_Coord_Lla     stLla;           // 현재 위경도 (출력용)
} ST_Target;

typedef struct
{
    ST_Target    stPlatform;        // 속도 0 인 표적 카드
    INT32        nTargetCnt;
    ST_Target    astTarget[MAX_TARGET]; // 10
    DOUBLE64    dSimTime;           // 60 s
    DOUBLE64    dDt;                // 0.1 s
} ST_Scenario;
```

03. 과제 항목이 들어간 칸, 그리고 설계 결정

과제가 요구한 항목 하나에 칸 하나. 좌표 타입은 참고 소스 코드의 것, 단위는 라디안 · m · s

과제 요구 항목	구조체 칸	타입 · 단위
초기 위경도 · 고도	ST_Target.stInitLla	STRUCT_Coord_Lla [rad,rad,m]
동체 속도 V_heading	ST_Target.dVheading	DOUBLE64 [m/s]
자세각 roll · pitch · yaw	ST_Target.stInitAtt	STRUCT_Coord_Attitude [rad]
기동 Gravity Value	ST_Maneuver.dGravity	DOUBLE64 [g], 부호 = 방향
기동 TurnType 0 / 1 / 2 / 3	ST_Maneuver.nTurnType	INT32 TURN_xxx 매크로
기동 StartTime · EndTime	ST_Maneuver.dStartTime · dEndTime	DOUBLE64 [s]
표적 최대 10 개	ST_Scenario.astTarget[MAX_TARGET]	#define MAX_TARGET 10
표적마다 기동 최대 30 개	ST_Target.astManeuver[MAX_MANEUVER]	#define MAX_MANEUVER 30
ECEF 갱신 위치 · 속도	ST_Target.stPos · stVel	STRUCT_Coord_Rect [m],[m/s]

동적 할당 없음 — 배열 크기는 과제 조건 10 · 30 을 #define 으로

표적을 늘리려면 숫자 하나. 초기화 뒤에는 상태 칸만 바뀌므로 같은 입력이면 언제 돌려도 같은 결과.

① 입력 칸과 상태 칸을 나눴다

초기화 때 입력을 한 번 읽어 상태를 만들고, 그 뒤로는 상태만. 다시 실행해도 입력이 그대로다.

② 좌표 타입은 참고 소스 코드의 구조체 그대로

STRUCT_Coord_Lla · Rect · Attitude 를 칸으로 쓰니 좌표변환 함수에 바로 넣고 바로 받는다.

③ 프로그램 안은 라디안 · m · s

도(°) 는 값을 넣는 f_SetTarget 에서만 받아 DEG2RAD. 화면에 보일 때만 RAD2DEG.

④ 플랫폼도 카드 한 장 (속력 0)

ST_Scenario.stPlatform. 같은 초기화 · 같은 스텝 함수를 지난다. 배가 움직이는 조건이 오면 속력과 yaw 만 넣으면 된다.

03. 값 채우기 — f_SetTarget · f_AddManeuver · f_SetAssignment

각도는 도로 받아 함수 안에서 라디안으로. 체크박스를 켜면 발표용 기동 예시가 붙는다

```
VOID f_SetAssignment(ST_Scenario *pstScn, INT32 bWithManeuver)
{
    memset(pstScn, 0, sizeof(*pstScn));
    pstScn->dSimTime = 60.0;
    pstScn->dDt = 0.1;
    pstScn->nTargetCnt = 2;

    /*      위도      경도      고도      속도      roll      yaw      pitch */
    f_SetTarget(&pstScn->stPlatform, 32.0, 126.0, 0.0, 0.0, 0.0, 0.0, 0.0);
    f_SetTarget(&pstScn->astTarget[0], 32.125, 126.03, 0.0, 30.0, 0.0, 270.0, 0.0); // 대함
    f_SetTarget(&pstScn->astTarget[1], 32.12, 126.0, 300.0, 200.0, 0.0, 180.0, 0.0); // 대공

    if (bWithManeuver) // 발표용 기동 예시
    {
        f_AddManeuver(&pstScn->astTarget[0], -0.1, TURN_YAW, 20.0, 50.0); // 대함 좌선회
        f_AddManeuver(&pstScn->astTarget[1], 3.0, TURN_YAW, 10.0, 20.0); // 대공 3 g 우선회
        f_AddManeuver(&pstScn->astTarget[1], 2.0, TURN_PITCH, 35.0, 40.0); // 대공 기수 들기
    }

    // f_SetTarget : 각도는 도로 받아 라디안으로 저장 (프로그램 안은 라디안 · m · s)
    pstTgt->stInitLla.Lat = DEG2RAD(dLatDeg); ... pstTgt->stInitAtt.Yaw = DEG2RAD(dYawDeg);
    // f_AddManeuver : 목록 끝에 붙인다. nManeuverCnt >= MAX_MANEUVER 면 무시
}
```

	플랫폼	표적 1 대함	표적 2 대공
위도 · 경도	32.0 · 126.0	32.125 · 126.03	32.12 · 126.0
고도 m	0	0	300
속력 m/s	0	30	200
yaw	0	270 (서)	180 (남)

기동 예시 (화면의 체크박스)

대함 : -0.1 g yaw 20~50 s (음수 = 좌선회)

대공 : +3 g yaw 10~20 s, +2 g pitch 35~40 s

표적을 더 넣으려면 f_SetTarget 줄을 더 쓴다 (최대 10).

시간 60 s · 간격 0.1 s · 표적 2 개 — 과제 시뮬레이션 조건 그대로

궤적 모의 구현

- 1 한 스텝의 다섯 단계
- 2 초기화와 속도 변환 — `f_InitTarget` · `f_BodyVelToEcef`
- 3 한 스텝 함수 ①② — 기동 판단 · 자세 갱신
- 4 한 스텝 함수 ③④⑤ — 속도 · 위치 · 위경도
- 5 시나리오 전체 — `f_StartScenario` · `f_StepScenario`
- 6 결과 모으기와 화면 — `TargetSimUI` (MFC)

함수 하나에 한 장. 코드는 `target_sim.c` 그대로다.

04. 한 스텝의 다섯 단계

표적 하나가 0.1초 뒤로 가는 일. 함수 하나가 이 순서 그대로이고, 0.1 초마다 표적 수만큼 반복한다

표적 하나의 한 스텝 (0.1 초)

왼쪽부터 순서대로. ① ② 는 기동이 없으면 건너뛰고, ③ ④ ⑤ 는 매 스텝 반드시 한다.



① 에 필요한 것

기동 목록, 지금 시각 t

② 에 필요한 것

각속도 $\omega = n \cdot g / V$, Δt

③ 에 필요한 것

자세각 3개, 표적 위경도

④ 에 필요한 것

ECEF 위치·속도, $\Delta t = 0.1\text{ s}$

⑤ 에 필요한 것

ECEF → LLA 변환 함수

순서가 중요한 건 ③ ④ ⑤ — 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면. 플랫폼과 표적 모두 같은 순서

04. 초기화와 속도 변환 — f_InitTarget · f_BodyVelToEcef

6장의 회전 두 번이 열 줄. 좌표변환 함수에서 조심할 점 두 가지가 이 화면에 다 있다

① 동체 → NED

f_Trans_Body_To_Ned 에 [V, 0, 0] 과 자세각.
인자 순서는 Roll, Yaw, Pitch (헤더에 적힌 대로).

② NED → ECEF

f_Trans_Ned_To_Ecef 의 가운데 셋이 플랫폼 위치.
속도는 벡터라 여기에 0. 위경도는 표적 자신의 것.

초기화의 함정

f_Trans_Lla_To_Ecef 는 (경도, 위도, 고도) 순서.
바꿔 놓으면 북위 54° 동경 212° 가 나온다.

검산 : $|v_{ecef}| = 200.000000 \text{ m/s}$

실수로 플랫폼 위치를 더하면 6,372 km/s

```
// 동체 속력 [V, 0, 0] 을 ECEF 속도로. 회전 두 번, 평행이동 없음
static STRUCT_Coord_Rect f_BodyVelToEcef(const ST_Target *pstTgt)
{
    STRUCT_Coord_Rect stNed;

    // 1) 동체 → NED : 자세각으로 회전 (인자 순서 Roll, Yaw, Pitch)
    stNed = f_Trans_Body_To_Ned(pstTgt->dVheading, 0.0, 0.0,
                                pstTgt->stAtt.Roll, pstTgt->stAtt.Yaw, pstTgt->stAtt.Pitch);

    // 2) NED → ECEF : 표적 자신의 위경도로 회전.
    // 속도는 벡터라 플랫폼 위치 인자(가운데 셋) 에 0
    return f_Trans_Ned_To_Ecef(stNed.x, stNed.y, stNed.z, 0.0, 0.0, 0.0,
                                pstTgt->stLla.Lat, pstTgt->stLla.Lon);
}

// 초기화 : 입력 칸 → 상태 칸. f_Trans_Lla_To_Ecef 는 인자 순서가 (경도, 위도, 고도)
static VOID f_InitTarget(ST_Target *pstTgt)
{
    pstTgt->stPos = f_Trans_Lla_To_Ecef(pstTgt->stInitLla.Lon,
                                        pstTgt->stInitLla.Lat, pstTgt->stInitLla.Alt);

    pstTgt->stAtt = pstTgt->stInitAtt;
    pstTgt->stLla = pstTgt->stInitLla;
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);
}
```

04. 한 스텝 함수 f_StepTarget — ①② 기동 판단과 자세 갱신

기동 목록을 훑어 지금 시각의 구간을 찾고, 그 축을 $\omega \times 0.1 \text{ s}$ 만큼 돌린다. 구간 밖이면 아무것도 안 한다

```
static VOID f_StepTarget(ST_Target *pstTgt, DOUBLE64 dTime, DOUBLE64 dDt)
{
    INT32 i;

    // 1) 기동 판단, 2) 자세 갱신 : 각속도  $\omega = n \cdot g / V$ 
    for (i = 0; i < pstTgt->nManeuverCnt; i++)
    {
        const ST_Maneuver *pstMan = &pstTgt->astManeuver[i];

        if ((dTime + 1e-6 >= pstMan->dStartTime) && (dTime + 1e-6 < pstMan->dEndTime)
            && (pstTgt->dVheading > 0.0))
        {
            DOUBLE64 dAng = pstMan->dGravity * G_FORCE / pstTgt->dVheading * dDt;

            if (pstMan->nTurnType == TURN_ROLL)    pstTgt->stAtt.Roll += dAng;
            else if (pstMan->nTurnType == TURN_YAW) pstTgt->stAtt.Yaw += dAng;
            else if (pstMan->nTurnType == TURN_PITCH) pstTgt->stAtt.Pitch += dAng;
            break;
            // 먼저 적용한 기동 하나만
        }
    }
    // ... ③ ④ ⑤ 는 다음 장
```

구간 판정 : $\text{Start} \leq t < \text{End}$

+1e-6 은 0.1 을 거듭 더할 때 생기는 부동소수점 오차 여유.
시작 10.0 인 기동이 한 스텝 늦지 않게.

각속도 한 줄 : $d\text{Ang} = n \cdot G_FORCE / V \cdot dt$

G_FORCE 9.80665 는 Define.h 의 상수.

3 g, 200 m/s, 0.1 s $\rightarrow$ 0.0147 rad = 0.84°.

축 선택과 break

TurnType 대로 Roll · Yaw · Pitch 하나에 더한다.

break — 겹치면 먼저 적용한 기동 하나만.

속력 0 이면 건너뛰

플랫폼도 같은 함수를 지나는데 $\omega = n \cdot g / V$ 의 분모가 V.
속력 0 인 카드는 기동 판단을 건너뛰고 자리만 지킨다.
끝난 뒤 자세 유지 = 안 건드리는 것.

기동 하나 = { Gravity Value, TurnType, StartTime, EndTime } 표적마다 최대 30 개



04. 한 스텝 함수 f_StepTarget — ③④⑤ 속도 · 위치 · 위경도

매 스텝 반드시 하는 셋. 자세가 정해졌으니 속도, 속도가 있으니 위치, 위치가 있으니 새 위경도

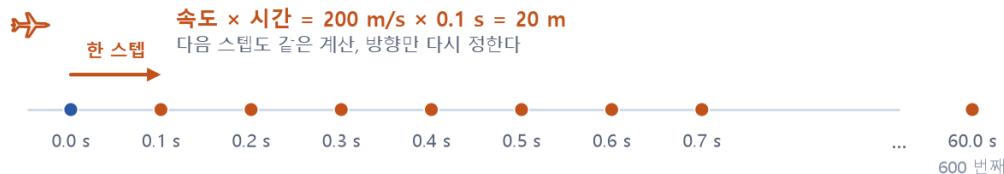
```
static VOID f_StepTarget(ST_Target *pstTgt, DOUBLE64 dTime, DOUBLE64 dDt)
{
    // ... ① ② 기동 판단 · 자세 갱신 (앞 장)

    // 3) 속도 변환 (표적 자신의 현재 위경도 기준 NED)
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);

    // 4) 위치 적분 (ECEF)
    pstTgt->stPos.x += pstTgt->stVel.x * dDt;
    pstTgt->stPos.y += pstTgt->stVel.y * dDt;
    pstTgt->stPos.z += pstTgt->stVel.z * dDt;

    // 5) LLA 변환. 출력용이자 다음 스텝의 NED 기준
    pstTgt->stLla = f_Trans_Ecef_To_Lla(pstTgt->stPos.x, pstTgt->stPos.y, pstTgt->stPos.z);
}
```

자동차로 치면 : 시속 720 km 로 0.1 초 가면 20 m. 그걸 600 번 더하면 12 km.
속도의 방향은 매 스텝 다시 계산하므로 (앞 장의 회전 두 번), 곡선도 이 방식으로 그려진다.



$$\text{새 위치} = \text{지금 위치} + \text{속도} \times 0.1 \text{ s}$$
$$p(t + \Delta t) = p(t) + v(t) \cdot \Delta t \quad \text{한 스텝 안에서는 속도가 일정하므로 이 한 줄로 충분하다}$$

③ 속도 변환 — 매 스텝 다시

자세가 안 바뀌어도 부른다.
표적의 위경도가 바뀌면 NED 축이 바뀌고 ECEF 속도 성분도 바뀐다.

④ 위치 적분 — 세 줄

$p \leftarrow p + v \times dt$ 를 ECEF x, y, z 에.
대공 200 m/s 면 한 스텝 20 m, 60 초에 12,000 m.

⑤ LLA 변환 — 출력이자 다음 수평면

f_Trans_Ecef_To_Lla 로. 이 값이 기록되고,
다음 스텝 ③ 의 NED 기준이 된다 (7장의 곡률 처리).

60 s 뒤 고도 300.019 m

한 스텝 20 m 접선 $\rightarrow 20^2/2R$ 썩 뜨고 60 초 동안 쌓여 1.9 cm.
이론값과 같다

04. 시나리오 전체 — f_StartScenario · f_StepScenario

표적 하나의 함수를 플랫폼 + 표적 전부로 넓힌 함수 둘. 코어의 시간 축은 이 둘이 전부다

```
// 시뮬레이션 시작 : 입력 칸으로 상태 칸을 채운다 (한 번)
VOID f_StartScenario(ST_Scenario *pstScn)
{
    INT32 i;

    f_InitTarget(&pstScn->stPlatform);
    for (i = 0; i < pstScn->nTargetCnt; i++)
    {
        f_InitTarget(&pstScn->astTarget[i]);
    }
}

// 시각 dTime 에서 dDt 만큼 전체를 한 스텝 진행 (0.1 초마다 불린다)
VOID f_StepScenario(ST_Scenario *pstScn, DOUBLE64 dTime)
{
    INT32 i;

    f_StepTarget(&pstScn->stPlatform, dTime, pstScn->dDt);
    for (i = 0; i < pstScn->nTargetCnt; i++)
    {
        f_StepTarget(&pstScn->astTarget[i], dTime, pstScn->dDt);
    }
}
```

플랫폼 → 표적 순서로 같은 함수

플랫폼은 속력 0 인 카드라 자리만 지킨다.
표적 수 nTargetCnt 만큼 (최대 10).

시각 인자 $dTime = k \times dt$

위치 적분은 시각을 몰라도 되지만
기동 판단은 지금이 어느 구간인지 알아야 한다.

부르는 순서 (UI 든 콘솔이든 같다)

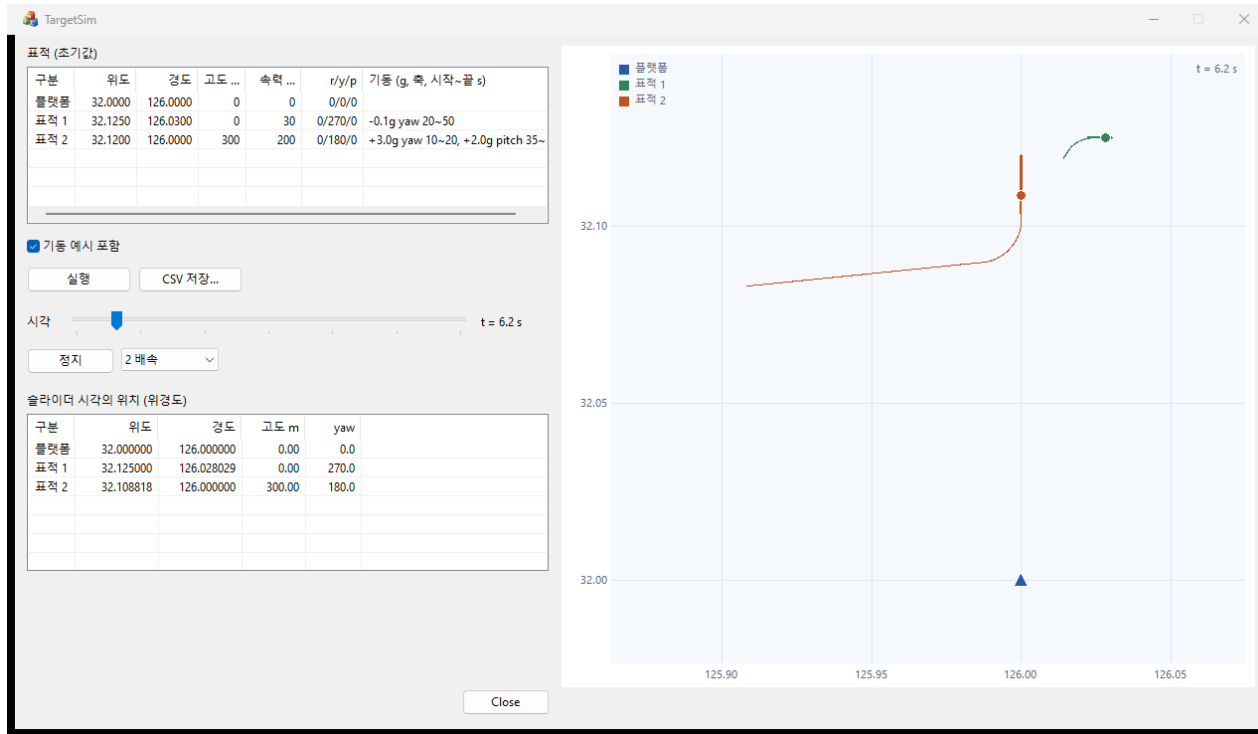
1 f_SetAssignment(&scn, bMan)	과제 조건
2 f_StartScenario(&scn)	한 번
3 f_StepScenario(&scn, t)	0.1 초마다

매 스텝 뒤 stLla 를 읽으면 그게 출력이다.

플랫폼과 표적이 같은 함수를 지난다 — 표적이 열 개로 늘어도 함수는 그대로

04. 결과 모으기와 화면 — TargetSimUI (MFC)

MFC 는 계산이 없다. 코어가 낸 위경도를 모아 지도 · 표 · CSV 로 보여 준다



재생 중인 화면 (t = 6.2 s). 점에 붙은 짧은 선이 진행 방향.

① 시나리오를 돌리며 위경도를 모은다

[실행] : f_StartScenario 한 번, f_StepScenario 를 0.1 초마다.
매 스텝의 위도 · 경도 · 고도 · yaw 를 표적별로 기록한다.

② 지도

기록한 위경도를 그린다. 위도 1° 가 경도 1° 보다 $1/\cos(\text{위도})$ 배 길어 축 비율을 맞춘다.
시각 슬라이더 · 재생으로 임의 시각의 위치를 본다.

③ 표와 CSV

슬라이더 시각의 위경도 표.
[CSV 저장] 은 t, id, 위도, 경도, 고도, yaw 를 한 줄씩 — 과제가 요구한 LLA 출력.

지도의 축 비율을 안 맞추면 선이 원이 타원으로 보인다. 뒤의 파이썬 그림은 이 CSV 로 그렸다.

시뮬레이션 결과

1 실행 화면 — 과제 조건

2 궤적과 고도

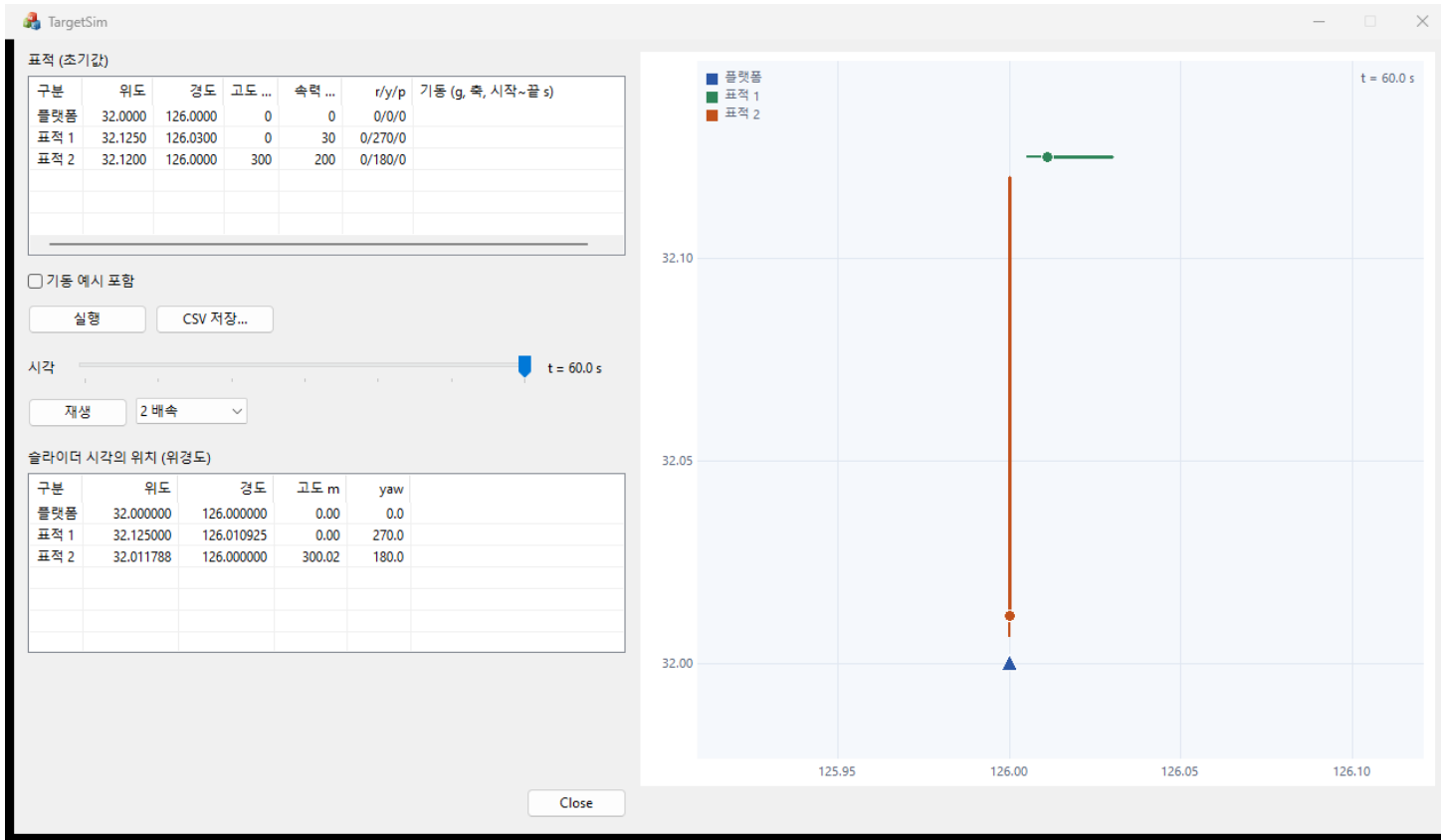
3 기동 시나리오

4 검증

화면의 값 = CSV 의 값 = 파이썬 그림의 값. 같은 코드가 만든 같은 숫자다.

05. 실행 화면 — 과제 조건, $t = 60\text{ s}$

왼쪽 위 표적 카드의 입력 칸, 오른쪽 궤적 지도, 왼쪽 아래 슬라이더 시각의 상태 칸



대공 표적 (주황)

60 s 뒤 32.011788°N, 126.0°E, 고도 300.02 m
 플랫폼에서 1,341 m 앞, 고각 1.2° → 12.9°
 정남으로 내려와 삼각형 바로 위에서 끝난다

대함 표적 (초록)

60 s 뒤 32.125°N, 126.010925°E
 서쪽으로 1.8 km. 선 길이 차이 = 속력 차이

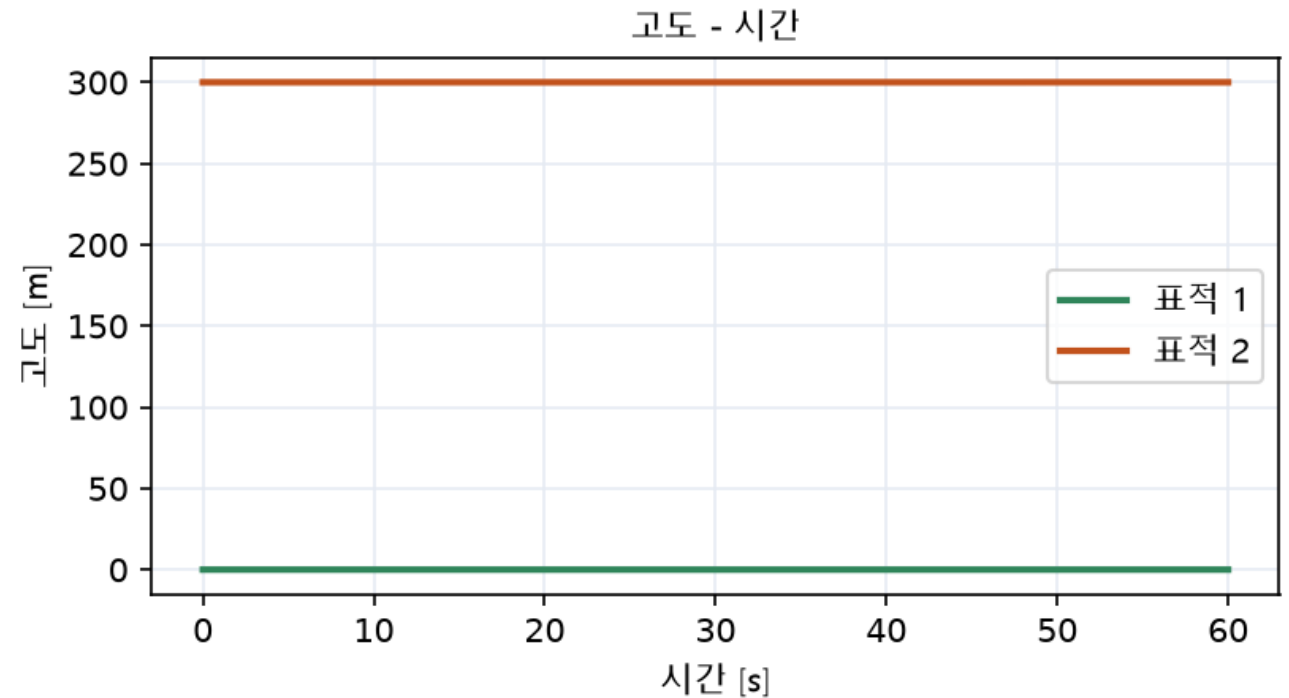
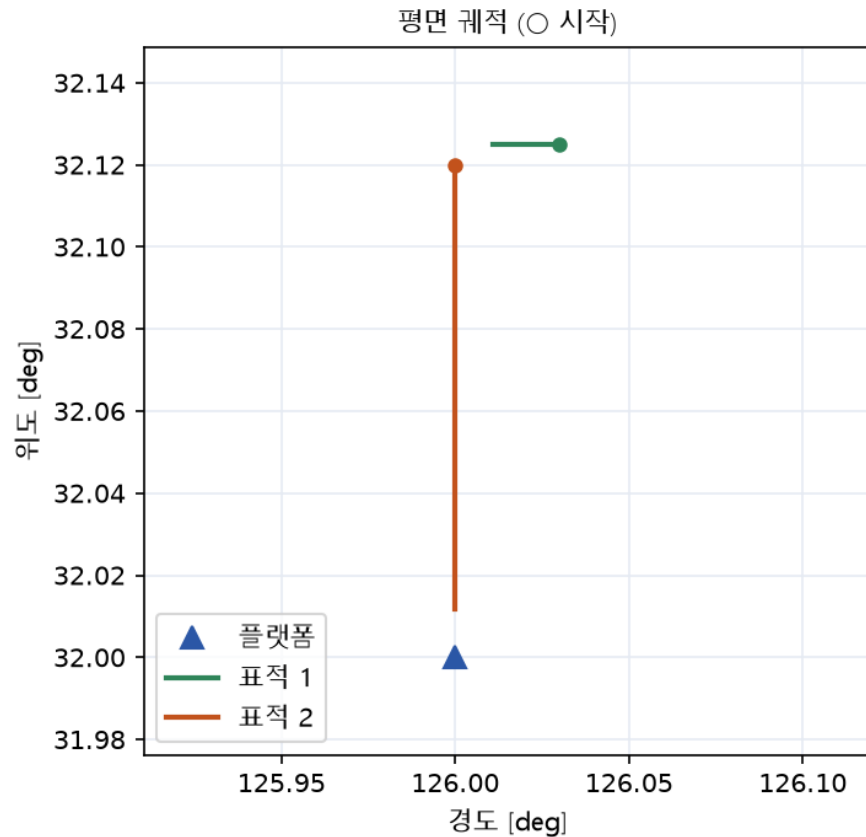
플랫폼 (파랑 삼각형)

32.0°N 126.0°E 에서 자리를 지킨다.
 같은 스텝 함수를 지나지만 속력이 0.

상태 칸의 값이 그대로 [CSV 저장] 으로 나간다 — 과제 요구 "시뮬레이션 결과를 LLA 로 출력"

05. 궤적과 고도 — CSV 를 파이썬으로

평면 궤적은 축 비율을 맞춰서, 고도는 평평해야 한다 (지구 곡률 처리의 증거)



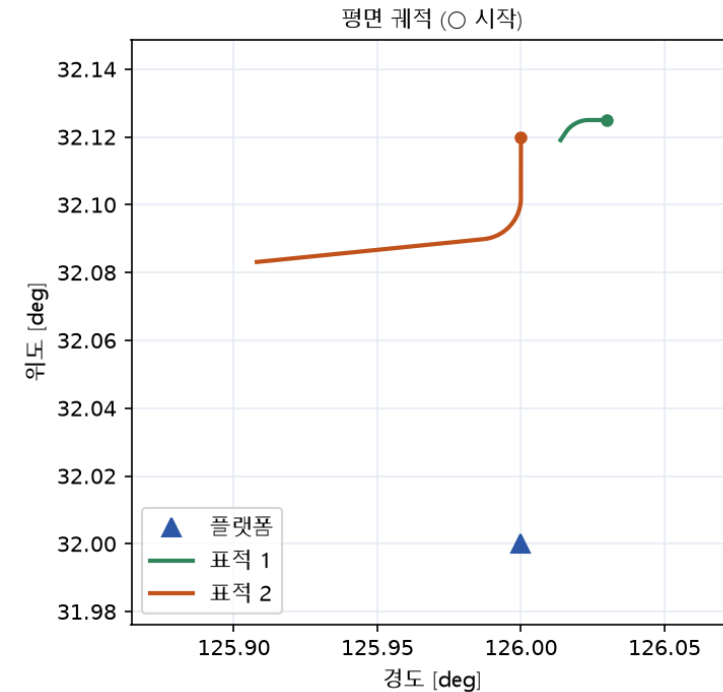
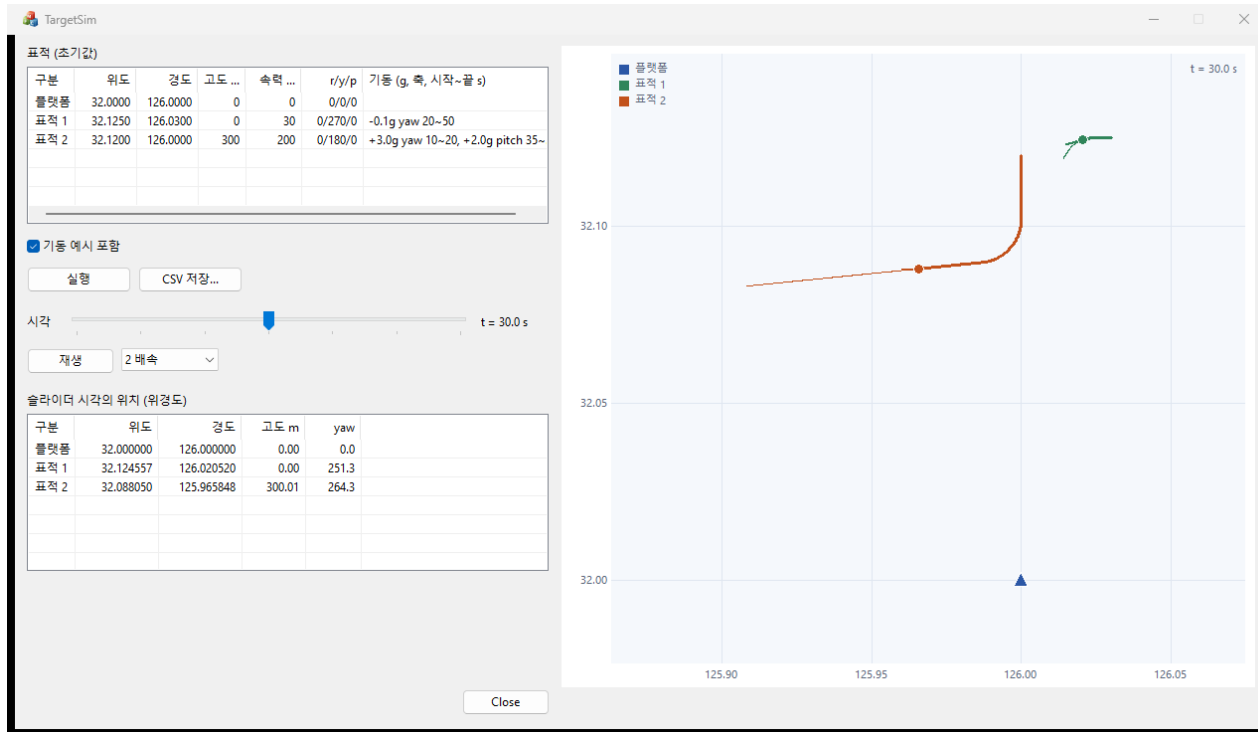
60 s 뒤 고도 300.019 m — 2 cm 의 정체

한 스텝에 20 m 접선으로 가서 $20^2 / 2R$ 씩 뜨고, 60 초 동안 쌓여 1.9 cm. 이론값과 같다.

매 스텝 수평면을 다시 잡지 않았다면 이 선이 300 → 311 m 로 기울었을 것

05. 기동 시나리오 — 3 g 우선회, 기수 들기, 좌선회

체크박스를 켜고 실행. 왼쪽은 $t = 30$ s 화면, 오른쪽은 CSV 로 그린 궤적



대공 3 g 우선회 (10~20 s)

10초에 84.3° 돌아 서쪽으로.

궤적 세 점 외접원 1,359.5 m vs $V^2/(n \cdot g)$ 1,359.6 m

대공 2 g 기수 들기 (35~40 s)

5초에 28.1° 올라간 뒤 그 각으로 20초.

60 s 에 고도 2,428.7 m

대함 -0.1 g 좌선회 (20~50 s)

30초에 -56.2° , 남서쪽으로.

반경 917.8 m vs 이론 917.7 m

05. 검증 — 다섯 가지로 확인

좌표변환은 틀려도 그럴듯한 값이 나온다 (과제 2의 교훈). 저장한 CSV 로 계산했다

항목	어떻게 확인하나	기대값	결과
① 이동 거리	ECEF 위치 차이의 합 vs 속도 × 시간	$200 \times 60 = 12,000 \text{ m}$	12,000.000 m (차 1e-9 m)
② 속도 보존	매 스텝 $ v_{\text{ecef}} $ vs 입력 속도	200 m/s	최소·최대 200.000000 (편차 1e-14)
③ 고도 유지	수평 비행 60 s 뒤 고도	300 m	300.019 m (이론 +1.9 cm)
④ 선회 반경	기동 시작·중간·끝 세 점의 외접원 vs $V^2/(n \cdot g)$	3 g : 1,359.6 m	1,359.5 m (−0.01 %)
⑤ 위경도 왕복	LLA → ECEF → LLA → ECEF 거리	1 mm 미만	3e-5 m

왜 미리 정하나

코드가 내놓을 값을 먼저 알고 있어야 코드가 맞는지 알 수 있다.
①②③⑤는 손계산한 값과 같고, ④는 기동 코드가 붙은 뒤 잦다.

한 번 틀리면 바로 드러나는 실수들

경도·위도 인자 순서 바꿈 → 북위 54° 동경 212°. 속도에 플랫폼 위치 더함 → 6,372 km/s.
수평면을 한 번만 잡음 → 고도 311 m.

다섯 칸이 채워졌으니 궤적이 "그림" 이 아니라 "결과" 다

정리

- 1 좌표변환 코드 : 함수 넷 — 경도 먼저 · Roll, Yaw, Pitch 순서 · 속도엔 플랫폼 위치 0 · 각도는 라디안
- 2 구조체 : 기동 · 표적 · 시나리오 셋. 표적은 입력 칸 다섯 + 상태 칸 넷, 과제 항목이 칸에 하나씩
- 3 플랫폼도 속도 0 인 표적 카드 — 초기화 · 스텝 함수가 하나로 통일된다
- 4 구현 : 기동 판단 → 자세 갱신 → 속도 변환 (회전 두 번) → 위치 적분 (ECEF) → LLA 변환, 0.1 초마다
- 5 LLA 변환은 출력이자 다음 스텝의 수평면 — 이 순서가 지구 곡률을 처리한다 (고도 300.019 m)
- 6 결과 : 대공 12 km 정면 접근 · 고각 $1.2^{\circ} \rightarrow 12.9^{\circ}$, 대함 1.8 km 서진, 3 g 선회 반경 1,359.5 m
- 7 검증 다섯 : 거리 · 속도 · 고도 · 선회 반경 · 왕복 — 전부 이론값과 일치

질문 환영합니다

코드 · 문서 : [radar-sw-study / Part3_TargetSim](#)